

HackNEX — Technical Architecture Document

Document Version: 1.0

Project: HackNEX

Organizer: NEXUS Club, Karunya Institute of Technology and Sciences

Event Mode: Offline

Event Dates: October 7–9, 2026

Registration Period: September 8–October 1, 2026

Expected Participants: 1,500+

Team Size: Exactly 4 participants

Registration Fee: ₹600 per team

Development Model: Solo Developer

Repository: [InfantBosco/Hackathon-web](#)

1. Document Purpose

This Technical Architecture Document defines **how the HackNEX website will be designed, developed, deployed, secured, and maintained.**

The document translates the previously finalized requirements into a technical implementation plan covering:

- Frontend architecture
- Backend architecture
- Database architecture
- Authentication
- Email verification
- Team registration
- Payment processing
- Payment verification
- Cloudinary media management
- Email communication
- Admin panel
- Security
- Scalability
- Deployment
- CI/CD
- Monitoring
- Testing
- Project structure

The architecture is designed specifically for a **solo developer** while providing sufficient capacity for **1,500+ participants**.

2. System Goals

The architecture must provide:

1. A premium hackathon website.
 2. Fast public-page loading.
 3. Secure authentication.
 4. Mandatory email verification.
 5. Exactly four participants per team.
 6. Captain-controlled registration.
 7. Prevention of duplicate participants.
 8. Automatic payment verification.
 9. Confirmation only after verified payment.
 10. Reliable email communication.
 11. Secure participant data storage.
 12. Admin management for approximately 15 administrators.
 13. Cloudinary-based media delivery.
 14. Scalability beyond the expected 1,500 participants.
 15. Simple enough maintenance for one developer.
-

3. Architecture Principles

The following principles will guide development.

3.1 Security First

Authentication, authorization, validation and payment verification must be performed server-side.

3.2 Separation of Concerns

Frontend, backend, database, payment, media and email responsibilities must remain separate.

3.3 Backend as Source of Truth

The frontend must never be trusted for:

- Payment status
- Registration status
- User permissions
- Team size
- Participant uniqueness

3.4 API-First Design

The frontend communicates with the backend through documented APIs.

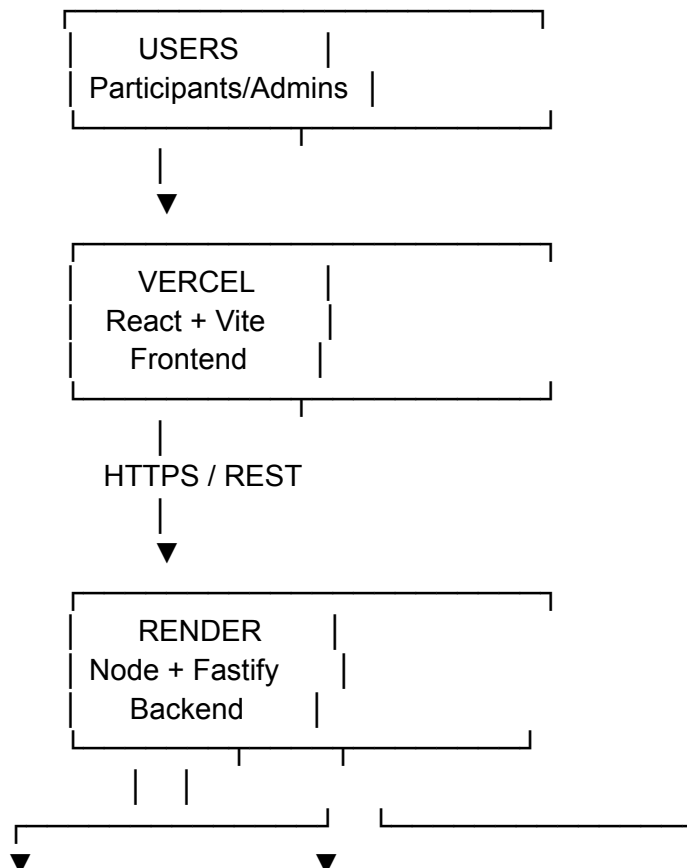
3.5 Modular Architecture

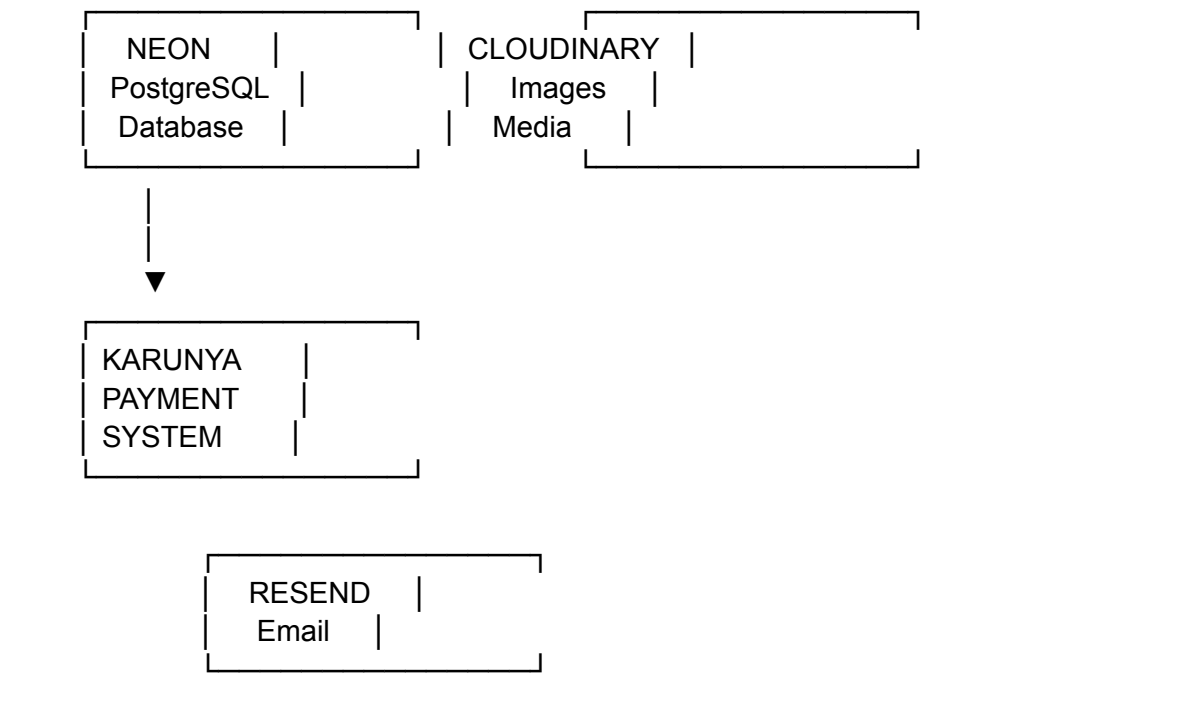
Payment, email and media integrations must be replaceable without redesigning the entire application.

3.6 Minimum Necessary Complexity

Because the project is developed by one person, unnecessary infrastructure should be avoided.

4. High-Level Architecture





5. Technology Stack

5.1 Frontend

Technology	Purpose
React	UI framework
TypeScript	Type safety
Vite	Build tooling
Tailwind CSS	Styling
shadcn/ui	Base UI components
21st.dev	Premium UI components
React Router	Routing
React Hook Form	Form management
Zod	Form validation
Framer Motion	Animations
Axios	API communication

6. Backend Stack

Technology	Purpose
Node.js	Runtime
TypeScript	Type safety
Fastify	API framework
Prisma	ORM
PostgreSQL	Database
Better Auth	Authentication
Zod	Validation

Fastify's schema-based validation and serialization model will be used for API-level validation in addition to frontend validation. [Fastify Validation and Serialization documentation](#)

7. Infrastructure Stack

Service	Purpose
Vercel	Frontend hosting
Render	Backend hosting
Neon	PostgreSQL hosting
Cloudinary	Image storage/CDN
Resend	Transactional email
Sentry	Error monitoring
Vercel Analytics	Website analytics

GitHub Source control
GitHub Actions CI/CD

8. Why This Architecture?

The expected scale is approximately:

1,500+ participants

With exactly four participants per team, this represents approximately:

375+ teams

The architecture should nevertheless accommodate considerably more records without requiring redesign.

A reasonable initial capacity target is:

5,000+ participant records

The application is not expected to require a complex microservices architecture.

Therefore, the recommended architecture is:

Modular Monolith + Managed Services

rather than:

Microservices + Kubernetes + multiple independently deployed services.

This dramatically reduces development and operational complexity for a solo developer.

9. Architectural Style

The backend will follow a **modular monolithic architecture**.

Fastify Application

- |
- |— Auth Module
- |— User Module
- |— Team Module
- |— Participant Module

- Registration Module
- Payment Module
- Email Module
- Media Module
- Content Module
- Admin Module
- Monitoring Module

All modules run within one backend application but remain logically separated.

10. Frontend Architecture

The frontend will follow a component-based architecture.

React Application

- Pages
 - Layouts
 - Components
 - Sections
 - Forms
 - Hooks
 - Services
 - Stores
 - Schemas
 - Types
 - Utilities
-

11. Frontend Routes

Recommended routes:

- /
- /about
- /domains
- /details
- /schedule
- /prizes
- /sponsors
- /host
- /faq

/login
/signup
/verify-email
/forgot-password
/reset-password

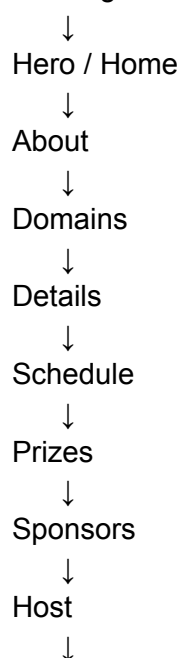
/register
/register/review
/register/payment
/register/success

/admin
/admin/teams
/admin/participants
/admin/payments
/admin/domains
/admin/schedule
/admin/prizes
/admin/sponsors
/admin/faqs
/admin/media
/admin/announcements

12. Public Website Architecture

The main page will use section-based navigation.

Loading Screen



FAQ

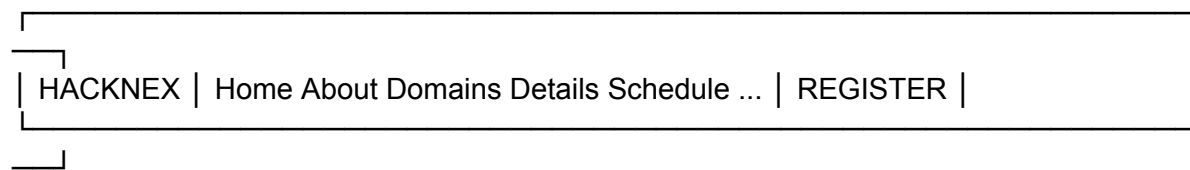


Footer

There will be no dedicated public Gallery section.

13. Navbar Architecture

Desktop:



Mobile:

HACKNEX



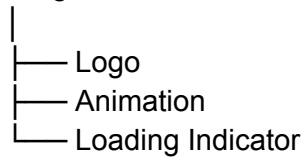
The navigation should be sticky.

The registration button should remain visually prominent.

14. Loading Screen Architecture

The application initially renders:

Loading Screen



Main Website

The loading animation must not artificially delay the website.

15. Registration CTA Architecture

Every major registration CTA should route to:

/register

If the user is not authenticated:

/register



Check Authentication



Not authenticated



/login

After authentication:

/login



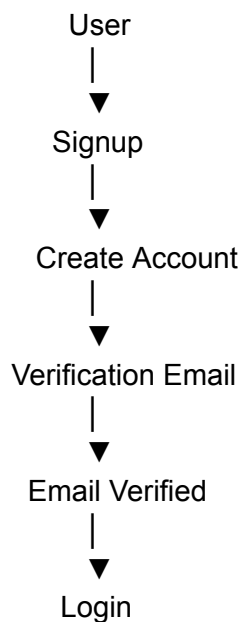
/register

16. Authentication Architecture

Authentication will use:

Better Auth

Authentication flow:



17. User Account Model

A user account represents the captain's authentication identity.

Basic information:

User

- id
- name
- email
- passwordHash / auth provider data
- emailVerified
- createdAt
- updatedAt

The authentication identity and participant record should be logically related but not treated as the same concept.

18. Why Separate User and Participant?

This distinction is important.

A:

User

is someone who can authenticate into the system.

A:

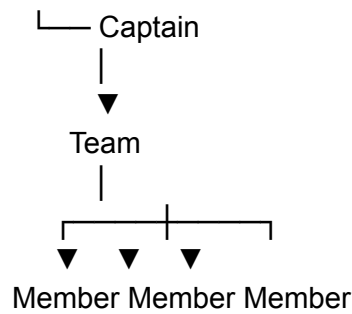
Participant

is someone registered for HackNEX.

The captain is both.

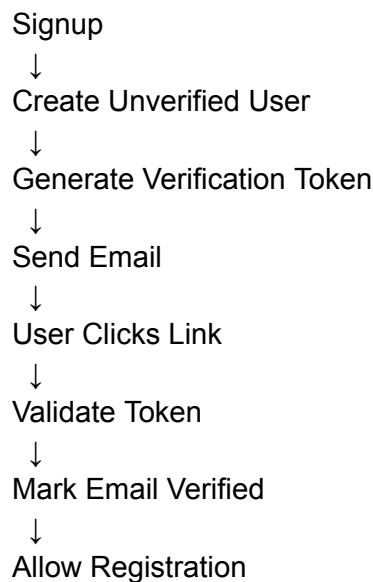
A team member may initially only be a participant.

User
|



19. Email Verification

Verification flow:



Verification tokens must:

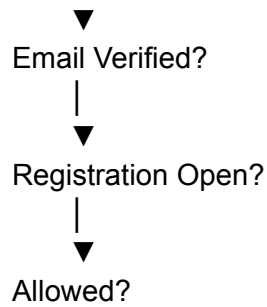
- Expire.
 - Be single-use.
 - Be securely generated.
 - Not expose passwords or sensitive data.
-

20. Registration Authorization

The backend must check:

Authenticated?

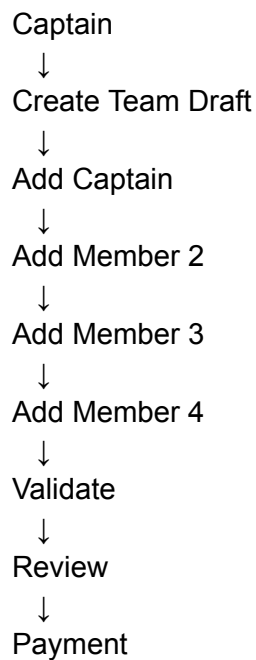




Only then should the registration API execute.

21. Team Registration Architecture

Registration should be implemented as a transaction-oriented process.



22. Registration State Machine

A registration should have explicit states.

Recommended:



↓
PAYMENT_PENDING
↓
PAYMENT_PROCESSING
↓
PAYMENT_SUCCESS
↓
PAYMENT_VERIFIED
↓
CONFIRMED

Failure states:

PAYMENT_FAILED
PAYMENT_EXPIRED
CANCELLED

23. Important Registration Rule

The database should never consider a registration successful merely because the frontend says:

paymentSuccess = true

Instead:

Frontend
↓
Payment System
↓
Backend
↓
Payment Verification
↓
Database
↓
CONFIRMED

24. Participant Uniqueness

The system must prevent a participant from registering in multiple teams.

The architecture should enforce this through database-level uniqueness where possible.

The unique participant identity should preferably be based on:

Verified email

and potentially phone number if the organizers decide this should also be unique.

25. Team Uniqueness

Team names should be unique.

Recommended database constraint:

UNIQUE(teamName)

Case-insensitive comparison should be considered so that:

HackNEX Warriors

and

hacknex warriors

are not treated as different teams.

26. Cross-College Teams

The system explicitly supports:

Team

- |
- | — KITS Student
- | — PSG Student
- | — CIT Student
- | — Other College Student

No database restriction should force all four members to belong to the same institution.

27. Cross-Department Teams

Participants can belong to different departments.

Example:

CSE

AI & DS

ECE

EEE

No department-level team restriction will be implemented.

28. Registration Form Architecture

The form should be broken into manageable steps.

Step 1

Team information.

Step 2

Captain.

Step 3

Member 2.

Step 4

Member 3.

Step 5

Member 4.

Step 6

Review.

Step 7

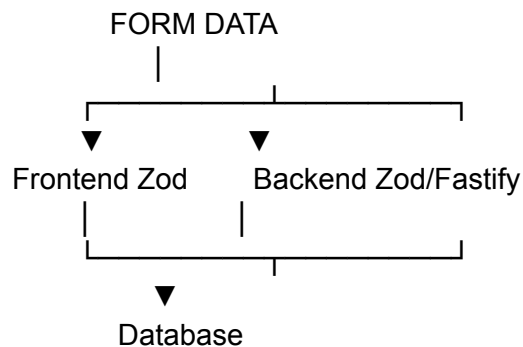
Payment.

Step 8

Confirmation.

29. Form Validation

Two layers:



Frontend validation improves UX.

Backend validation provides actual security.

30. Participant Data Model

Each participant contains:

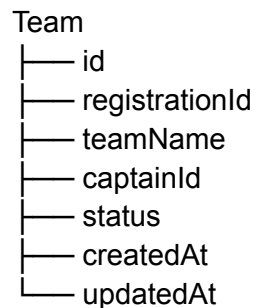
Participant

- id
- teamId
- name
- email
- phone
- college
- department
- yearOfStudy
- linkedinUrl
- foodPreference
- isCaptain
- createdAt
- updatedAt

Food preference:

VEG

31. Team Data Model



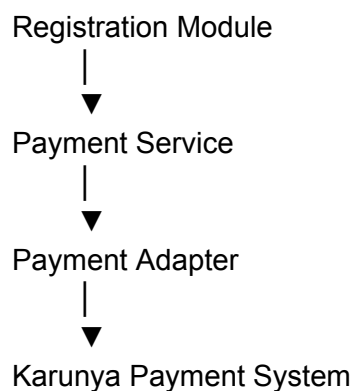
A team must have exactly four participant records before it can enter payment.

32. Payment Architecture

The payment architecture will use a **Payment Adapter Pattern**.

This is particularly important because the exact Karunya payment API details are not yet available.

Architecture:



If Karunya's implementation changes later, we modify the adapter rather than the entire registration system.

33. Payment Adapter

Conceptually:

PaymentProvider

```
|
|— createPayment()
|— getPaymentStatus()
|— verifyPayment()
|— handleWebhook()
```

Later:

KarunyaPaymentProvider

can implement these operations.

34. Payment Flow

Registration



Calculate Fee



₹600



Create Payment Record



Initiate Karunya Payment



User Completes Payment



Karunya System



Verification



Backend



Payment VERIFIED



Registration CONFIRMED

35. Payment Amount Security

The frontend must never be allowed to decide the amount.

Bad:

```
POST /payment
{
  "amount": 100
}
```

Instead:

```
Backend
  ↓
Registration
  ↓
Server determines fee
  ↓
₹600
```

The server is the source of truth.

36. Payment Records

Recommended fields:

```
Payment
├── id
├── registrationId
├── amount
├── currency
├── provider
├── transactionId
├── status
├── verifiedAt
├── createdAt
└── updatedAt
```

37. Payment Status

Recommended enum:

```
PENDING
INITIATED
PROCESSING
SUCCESS
```

VERIFIED
FAILED
EXPIRED
CANCELLED

The final implementation will map these states to the actual Karunya payment responses.

38. Payment Verification

There are two preferred mechanisms.

Preferred

Karunya
↓
Webhook
↓
HackNEX Backend
↓
Verify Transaction
↓
Database

Alternative

HackNEX Backend
↓
Karunya Verification API
↓
Transaction Status
↓
Database

The exact mechanism depends on the API provided by Karunya.

39. Payment Idempotency

Payment operations must be idempotent.

If Karunya sends the same payment notification three times:

Webhook
Webhook

Webhook

the database should still contain:

ONE VERIFIED PAYMENT

not three payments.

40. Registration Confirmation

Confirmation happens only after:

Payment = VERIFIED

Then:

Registration



CONFIRMED



Generate Registration ID



Queue Confirmation Email

41. Registration ID

Recommended format:

HNX-2026-000001

The exact format can be changed later.

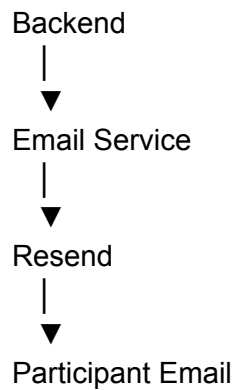
It should be unique.

42. Email Architecture

Email will use:

Resend

Architecture:



43. Required Emails

Authentication

- Email verification
- Password reset

Registration

- Registration confirmation
- Payment confirmation where applicable

Event

- Important announcements
 - Registration updates
-

44. Email Templates

Templates should be maintained separately from business logic.

emails/

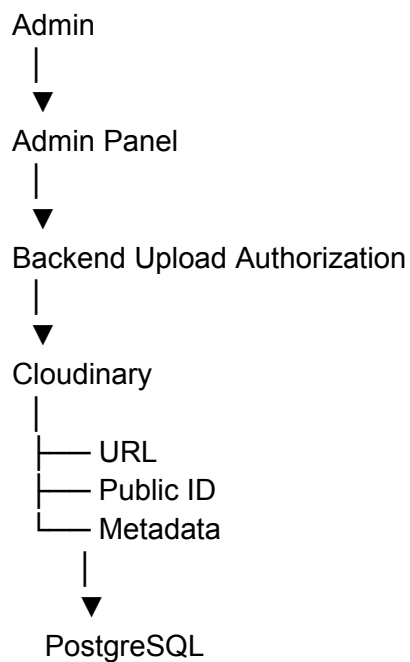
- verification
- password-reset
- registration-confirmed
- payment-confirmed
- announcement

This allows the design team to modify email appearance without modifying payment/registration logic.

45. Cloudinary Architecture

Cloudinary will manage website images.

Architecture:



46. Cloudinary Usage

Images include:

- Hero images
- About images
- Venue images
- Sponsor logos
- Organizer logos
- Other visual assets

No dedicated public gallery is required.

47. Media Database Model

Recommended:

Media

- |— id
- |— publicId
- |— secureUrl
- |— resourceType
- |— category
- |— title
- |— altText
- |— displayOrder
- |— isActive
- |— createdAt
- |— updatedAt

The actual image remains in Cloudinary.

48. Cloudinary Categories

HERO
ABOUT
VENUE
SPONSOR
ORGANIZER
OTHER

49. Image Optimization

Cloudinary should be configured for:

- Automatic format selection
- Automatic quality optimization
- Responsive sizing
- CDN delivery
- Appropriate transformations

This prevents the React application from serving unnecessarily large original images.

50. Admin Media Operations

Admins can:

- Upload
- Delete
- Replace
- Reorder
- Categorize
- Activate
- Deactivate

51. Admin Architecture

There will be approximately:

15 administrators

All administrators have the same:

Admin

role.

No separate finance/content/super-admin roles are required initially.

52. Admin Authentication

Admin access must use the same secure authentication foundation but require an administrative role.

Login
↓
Authenticated
↓
Check Role
↓
ADMIN?
↓
Allow Dashboard

The frontend hiding the admin route is **not sufficient**.

The backend must protect every admin API.

53. Admin Dashboard

Dashboard metrics:

Teams	Participants	
375+	1500+	
Confirmed	Pending	
Payments	Registrations	
Veg Participants	Non-Veg Participants	

54. Admin Modules

Admin Dashboard

Teams
Participants
Payments
Domains
Schedule
Prizes
Sponsors
FAQ
Media
Announcements

55. Admin Team Management

Admin can:

- Search teams
- View team details
- View captain
- View members
- View registration status

- View payment status
 - View registration ID
-

56. Admin Participant Management

Admin can:

- Search participants
 - Filter by college
 - Filter by department
 - Filter by year
 - Filter by food preference
 - View team
 - View participant details
-

57. Admin Payment Management

Admin can view:

- Team
- Registration ID
- Transaction ID
- Amount
- Payment status
- Verification status
- Payment date

Even though payment is automatically verified, administrators need visibility for reconciliation and troubleshooting.

58. Content Management

The admin panel can manage:

- Domains
- Schedule
- Prizes
- Sponsors

- [FAQ](#)
- [Announcements](#)
- [Media](#)

This avoids requiring code changes every time event information changes.

59. API Architecture

The API will use REST principles.

Base URL:

/api

60. Authentication API

POST /api/auth/signup
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/verify-email
POST /api/auth/resend-verification
POST /api/auth/forgot-password
POST /api/auth/reset-password
GET /api/auth/session

61. Public Content API

GET /api/hackathon
GET /api/domains
GET /api/schedule
GET /api/prizes
GET /api/sponsors
GET /api/faqs
GET /api/media

62. Registration API

POST /api/registrations

GET /api/registrations/:id
PUT /api/registrations/:id
POST /api/registrations/:id/submit

63. Payment API

POST /api/payments/create
GET /api/payments/:id
POST /api/payments/:id/verify
POST /api/payments/webhook

The final webhook endpoint will depend on Karunya's payment system.

64. Admin API

GET /api/admin/dashboard

GET /api/admin/teams
GET /api/admin/participants
GET /api/admin/payments

POST /api/admin/domains
PUT /api/admin/domains/:id
DELETE /api/admin/domains/:id

POST /api/admin/schedule
PUT /api/admin/schedule/:id
DELETE /api/admin/schedule/:id

POST /api/admin/prizes
POST /api/admin/sponsors
POST /api/admin/faqs

POST /api/admin/media
DELETE /api/admin/media/:id

65. API Response Standard

Successful response:

```
{
  "success": true,
  "data": {}
}
```

Error response:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid participant information"
  }
}
```

This keeps frontend API handling consistent.

66. Database Architecture

The primary database will be:

PostgreSQL

hosted through:

Neon

ORM:

Prisma

67. Core Database Entities

User

Session

Team

Participant

Registration

Payment

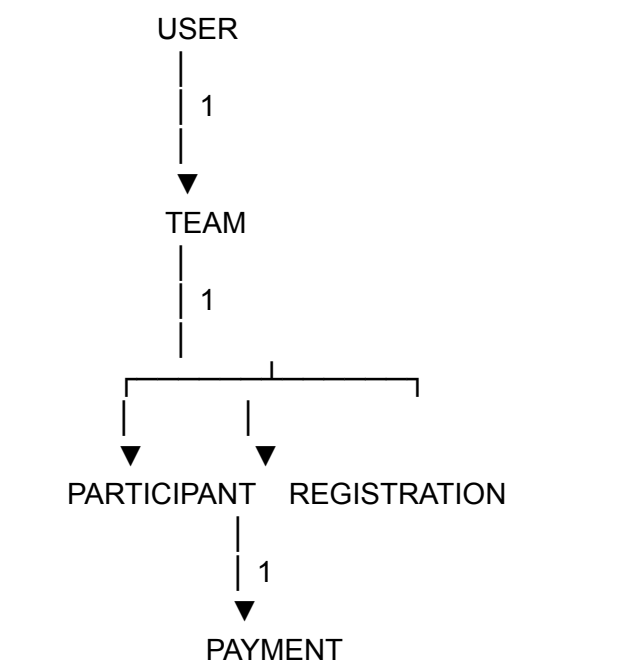
Media

Domain

Schedule

Prize

68. Entity Relationship Overview



ADMIN —————> AUDIT LOG

MEDIA —————> CLOUDINARY

DOMAIN
SCHEDULE
PRIZE
SPONSOR
FAQ
ANNOUNCEMENT

69. Database Relationship Rules

User → Team

One captain/user can have the relevant team registration.

Team → Participants

Exactly four participants after registration completion.

Team → Registration

One registration.

Registration → Payment

One or more payment attempts may technically exist, but only one successful/verified payment should finalize the registration.

70. Suggested Prisma Models

The actual Prisma schema will be developed separately, but conceptually:

User
Team
Participant
Registration
Payment
Media
Domain
ScheduleItem
Prize
Sponsor
FAQ
Announcement
AuditLog

71. Database Indexing

Indexes should exist for frequently searched fields.

Recommended:

User.email
Participant.email
Participant.phone
Participant.college
Participant.department

Team.teamName
Registration.registrationId
Registration.status
Payment.transactionId
Payment.status

This is important for the admin panel.

72. Database Transactions

Critical operations should use database transactions.

Example:

Create Team
+
Create 4 Participants
+
Create Registration

should succeed together or fail together.

This prevents incomplete registrations.

73. Registration Transaction

BEGIN TRANSACTION

Create Team

Create Captain
Create Member 2
Create Member 3
Create Member 4

Create Registration

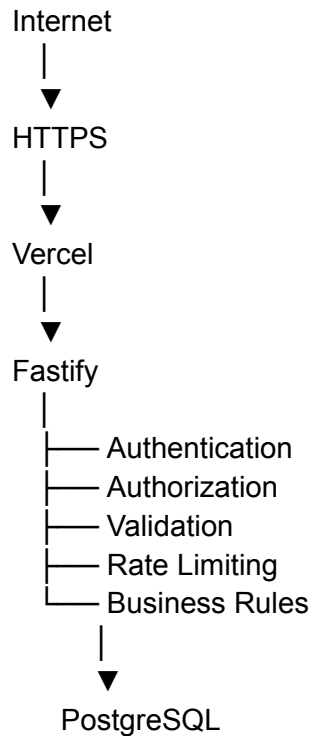
COMMIT

If one operation fails:

ROLLBACK

74. Security Architecture

Security layers:



75. Password Security

Passwords must never be stored as plain text.

Authentication framework handles secure password processing.

The application should never log passwords.

76. Environment Variables

Example:

```
DATABASE_URL=  
BETTER_AUTH_SECRET=
```

RESEND_API_KEY=

CLOUDINARY_CLOUD_NAME=

CLOUDINARY_API_KEY=

CLOUDINARY_API_SECRET=

PAYMENT_API_URL=

PAYMENT_API_KEY=

PAYMENT_SECRET=

SENTRY_DSN=

No secrets should exist in GitHub.

77. Frontend Environment Variables

Only public configuration should be exposed to the frontend.

Example:

VITE_API_URL=

VITE_CLOUDINARY_CLOUD_NAME=

Never expose:

PAYMENT_SECRET

DATABASE_URL

CLOUDINARY_API_SECRET

78. Rate Limiting

Rate limiting should be applied especially to:

Signup

Login

Forgot Password

Resend Verification

Payment APIs

Admin APIs

This reduces brute-force and abuse risks.

79. Input Security

All user-provided data must be validated.

Examples:

- Email
 - Phone
 - LinkedIn
 - Team name
 - College
 - Department
 - Year
 - Food preference
-

80. SQL Injection Protection

Prisma's parameterized query mechanisms should be used.

Raw SQL should be avoided unless there is a clear technical reason.

81. XSS Protection

User-provided content must never be rendered as unrestricted HTML.

This is particularly important for admin-managed:

- FAQ
 - Announcements
 - Domain descriptions
 - Sponsor descriptions
-

82. CSRF / Session Security

Authentication/session implementation must follow the selected authentication framework's secure configuration.

Cookies should use appropriate:

HttpOnly
Secure
SameSite

settings where applicable.

83. Admin Audit Logging

Important admin operations should create audit records.

Example:

Admin
↓
Deleted Media
↓
Timestamp
↓
AuditLog

Potential events:

- Participant modification
 - Team modification
 - Payment status modification
 - Media deletion
 - Content modification
-

84. Privacy

The system stores personal information.

Therefore:

- Privacy Policy should be published.
 - Access should be restricted.
 - Data should only be used for legitimate HackNEX purposes.
 - Sensitive data should not appear in logs.
 - Admin access should be controlled.
-

85. No Refund Policy

The registration process should explicitly display:

Registration fee: ₹600/team

and:

Registration fees are non-refundable.

The user must acknowledge the policy before proceeding to payment.

86. Scalability Architecture

Expected:

1,500+ participants

The architecture should support:

5,000+ participants

without fundamental redesign.

The main scaling strategy is:

Stateless Backend

+

Managed PostgreSQL

+

CDN

+

Cloudinary

+

Efficient Queries

87. Stateless Backend

The Fastify backend should remain as stateless as practical.

This makes horizontal scaling easier if required later.

88. Frontend Scaling

The frontend is deployed on Vercel.

Static assets can be distributed through CDN infrastructure.

Cloudinary handles image delivery.

Therefore, images do not consume backend server bandwidth.

89. Database Scaling

Neon PostgreSQL provides managed database infrastructure.

Database optimization will focus on:

- Indexing
 - Efficient queries
 - Pagination
 - Connection management
 - Transactions
-

90. Admin Pagination

Admin lists must not load all 1,500+ participants simultaneously.

Instead:

GET /participants?page=1&limit=25

Example:

Page 1 → 25

Page 2 → 25

...

91. Search

Server-side search should be implemented.

Example:

Search:

"Infant"

Database



Filtered Results

This avoids downloading all participant records to the browser.

92. Caching

Caching should initially be kept simple.

Potentially cache public content such as:

- Domains
- Schedule
- Prizes
- Sponsors
- FAQ

Registration and payment data should not rely on stale caches.

93. Redis Decision

Redis is **not required for V1**.

Reason:

- Only ~1,500+ participants expected.
- Solo developer.
- Additional infrastructure adds complexity.
- Most operations can be handled by PostgreSQL and managed services.

Redis can be introduced later if actual traffic requires:

- Queue processing
- Advanced rate limiting
- Distributed caching
- Background jobs

94. Email Processing

For the initial implementation, transactional emails can be sent through Resend.

If email volume or reliability requirements increase, a queue can later be introduced:

Backend

↓

Redis

↓

BullMQ

↓

Resend

This is deliberately kept as a future scaling layer.

95. Media Performance

Images should use:

- Cloudinary optimization
- Responsive dimensions
- Lazy loading
- CDN delivery
- Modern formats

Critical hero assets may be loaded earlier.

96. SEO Architecture

Public pages should include:

- Title
- Meta description
- Open Graph metadata
- Favicon
- Canonical metadata
- Sitemap

- Robots configuration

Important public search terms may include:

HackNEX

HackNEX 2026

Karunya hackathon

Coimbatore hackathon

NEXUS Club hackathon

97. Analytics

We will use:

Vercel Analytics

Potential measurements:

- Visitors
- Page views
- Registration CTA clicks
- Traffic sources
- Device categories

No unnecessary participant information should be sent to analytics systems.

98. Monitoring

Sentry will monitor:

- Frontend exceptions
- Backend exceptions
- API failures
- Registration failures
- Payment integration failures

Critical errors should trigger appropriate alerts.

99. Logging Architecture

Backend logs should capture:

Request
Response status
Error
Timestamp
Request ID

Sensitive information must be excluded.

Do not log:

Passwords
Authentication secrets
Payment credentials
API secrets

100. Deployment Architecture

Frontend

GitHub
↓
Vercel
↓
Production

Backend

GitHub
↓
Render
↓
Fastify

Database

Neon
↓
PostgreSQL

Media

Cloudinary

Email

Resend

101. CI/CD Architecture

Recommended workflow:

```
graph TD; Developer --> GitBranch[Git Branch]; GitBranch --> PullRequest[Pull Request]; PullRequest --> GitHubActions[GitHub Actions]; GitHubActions --> Lint; Lint --> TypeCheck[Type Check]; TypeCheck --> Tests; Tests --> Build; Build --> Merge; Merge --> Deployment;
```

102. Git Branching

Since this is a solo project, don't use an unnecessarily complicated Git workflow.

Recommended:

```
main
|
├── feature/frontend
├── feature/auth
├── feature/registration
├── feature/payment
├── feature/admin
└── feature/cloudinary
```

Merge tested features into `main`.

103. Project Repository

Repository:

InfantBosco/Hackathon-web

The repository should contain:

frontend/

backend/

docs/

or a structured monorepo.

104. Recommended Repository Structure

I recommend a monorepo:

hackathon-web/

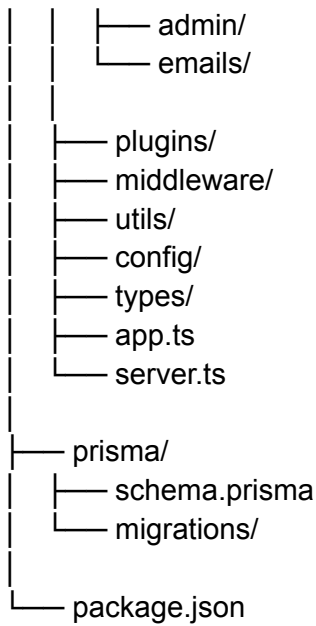
```
|
|--- apps/
|   |--- web/
|   |--- api/
|
|--- packages/
|   |--- shared/
|   |--- config/
|
|--- docs/
|
|--- .github/
|   |--- workflows/
|
|--- .gitignore
|--- package.json
|--- README.md
|--- pnpm-workspace.yaml
```

105. Frontend Folder Structure

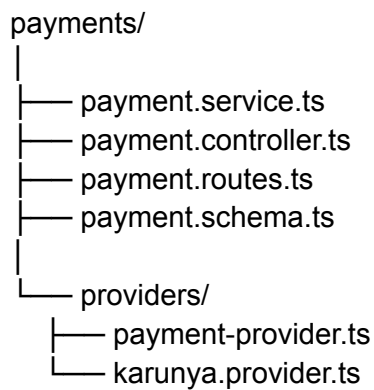
```
apps/web/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── ui/
│   │   ├── navbar/
│   │   ├── footer/
│   │   └── sections/
│   ├── pages/
│   │   ├── public/
│   │   ├── auth/
│   │   ├── registration/
│   │   └── admin/
│   ├── hooks/
│   ├── services/
│   ├── stores/
│   ├── schemas/
│   ├── types/
│   ├── utils/
│   ├── router/
│   └── main.tsx
├── public/
├── index.html
└── vite.config.ts
```

106. Backend Folder Structure

```
apps/api/
├── src/
│   └── modules/
│       ├── auth/
│       ├── users/
│       ├── teams/
│       ├── participants/
│       ├── registrations/
│       ├── payments/
│       ├── media/
│       └── content/
```

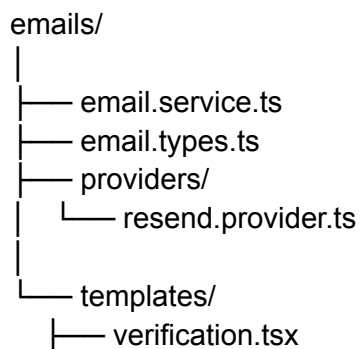


107. Payment Module Structure



This makes the future Karunya integration isolated.

108. Email Module Structure



- password-reset.tsx
- registration-confirmed.tsx
- payment-confirmed.tsx

109. Media Module Structure

```
media/
|
|— media.service.ts
|— media.controller.ts
|— media.routes.ts
|— media.schema.ts
|— cloudinary.service.ts
```

110. Testing Architecture

Testing will occur at multiple levels.

Unit

Vitest

Integration

API + database

End-to-End

Playwright

111. Critical E2E Test

The most important automated test should simulate:

```
Signup
↓
Email Verification
↓
Login
↓
```

Team Registration



4 Participants



Review



Payment



Payment Verification



Registration Confirmation



Email

112. Negative Tests

Test:

- Duplicate email
 - Duplicate participant
 - Duplicate team
 - Invalid phone
 - Invalid LinkedIn
 - Missing member
 - Five-member team
 - Three-member team
 - Unverified user
 - Expired verification
 - Failed payment
 - Duplicate payment
 - Unauthorized admin access
-

113. Load Testing

Before production, test:

100 concurrent users

250 concurrent users

500 concurrent users

The primary focus should be:

- Login
 - Registration APIs
 - Database writes
 - Public website traffic
 - Payment callback handling
-

114. Backup Architecture

Registration data is classified as:

Critical

Therefore:

Neon PostgreSQL



Automated Backup



Recovery Procedure

The backup and retention configuration should be verified before registration opens.

115. Disaster Recovery

Potential failures:

Frontend failure

Backend failure

Database failure

Payment failure

Email failure

Cloudinary failure

The system should fail gracefully.

For example:

Email failure

Registration should not become invalid simply because the confirmation email temporarily fails.

Instead:

Registration = CONFIRMED

Email = FAILED

The email can be retried.

116. Payment Failure Recovery

If payment verification is temporarily unavailable:

Payment



Verification Pending



Retry Verification



Confirmed

The system must not incorrectly mark the team as failed without evidence.

117. Registration Deadline

The backend must enforce:

Registration Start



Registration Open



October 1



Registration Closed

The frontend countdown alone is not sufficient.

The backend must reject registration after the deadline.

118. Event Mode

The event is now officially:

Offline

Venue:

Karunya Institute of Technology and Sciences, Coimbatore

The public website should therefore communicate the physical venue clearly.

119. Venue Architecture

The venue section can contain:

- KITS name
- Location
- Campus image
- Map
- Directions link
- Institution website

The actual event location/building/room should be added when finalized.

120. Accessibility

The frontend should provide:

- Keyboard navigation
 - Semantic HTML
 - Alt text
 - Proper labels
 - Visible focus
 - Accessible forms
 - Sufficient contrast
 - Reduced-motion support
-

121. Performance Targets

Recommended targets:

Metric	Target
Public page load	Fast on 4G
API response	Generally <500 ms
Database query	Generally <200 ms
Image delivery	CDN optimized
Lighthouse Performance	Aim for 90+
Mobile usability	Fully responsive

These are engineering targets, not absolute guarantees.

122. Error Handling Strategy

All errors should be classified.

VALIDATION_ERROR
AUTHENTICATION_ERROR
AUTHORIZATION_ERROR
NOT_FOUND
DUPLICATE_RESOURCE
PAYMENT_ERROR
PAYMENT_VERIFICATION_ERROR
INTERNAL_SERVER_ERROR

The frontend should convert technical errors into user-friendly messages.

123. Example Error Flow

Database Error
↓
Backend catches error
↓
Logs error with Sentry
↓
Returns safe error response
↓
Frontend displays:

"Something went wrong.
Please try again."

No database stack trace should be exposed to users.

124. Production Security Checklist

Before launch:

- ☐ HTTPS enabled
 - ☐ Production secrets configured
 - ☐ Database access restricted
 - ☐ Admin authorization tested
 - ☐ Rate limiting enabled
 - ☐ Authentication tested
 - ☐ Email verification tested
 - ☐ Password reset tested
 - ☐ Payment verification tested
 - ☐ Duplicate payment tested
 - ☐ Duplicate participant tested
 - ☐ Input validation tested
 - ☐ CORS configured
 - ☐ Security headers configured
 - ☐ Error logging enabled
 - ☐ Backups verified
-

125. Production Registration Checklist

Before opening registration:

- ☐ Registration dates correct
- ☐ ₹600 fee configured
- ☐ Exactly 4 members enforced
- ☐ Team-name uniqueness enabled
- ☐ Participant uniqueness enabled
- ☐ Cross-college teams supported
- ☐ Cross-department teams supported
- ☐ Food preferences working
- ☐ LinkedIn validation working
- ☐ Email verification working
- ☐ Payment working
- ☐ Payment verification working

- ☐ Confirmation email working
 - ☐ Registration ID generation working
-

126. Production Admin Checklist

- ☐ 15 admin accounts created
 - ☐ Admin authentication tested
 - ☐ Team search tested
 - ☐ Participant search tested
 - ☐ Payment dashboard tested
 - ☐ Export tested
 - ☐ Content management tested
 - ☐ Media upload tested
 - ☐ Media deletion tested
 - ☐ Audit logs tested
-

127. Important External Dependency

The architecture currently contains one intentionally abstract component:

Karunya Payment Provider

because the exact Karunya payment API/webhook mechanism has not yet been supplied.

Therefore the architecture uses:

Payment Adapter Pattern

This allows us to finalize the rest of the system now.

Once Karunya provides:

- Payment URL
- API documentation
- Authentication mechanism
- Transaction ID format
- Verification API
- Webhook details
- Success/failure response format

only the payment provider implementation needs to be finalized.

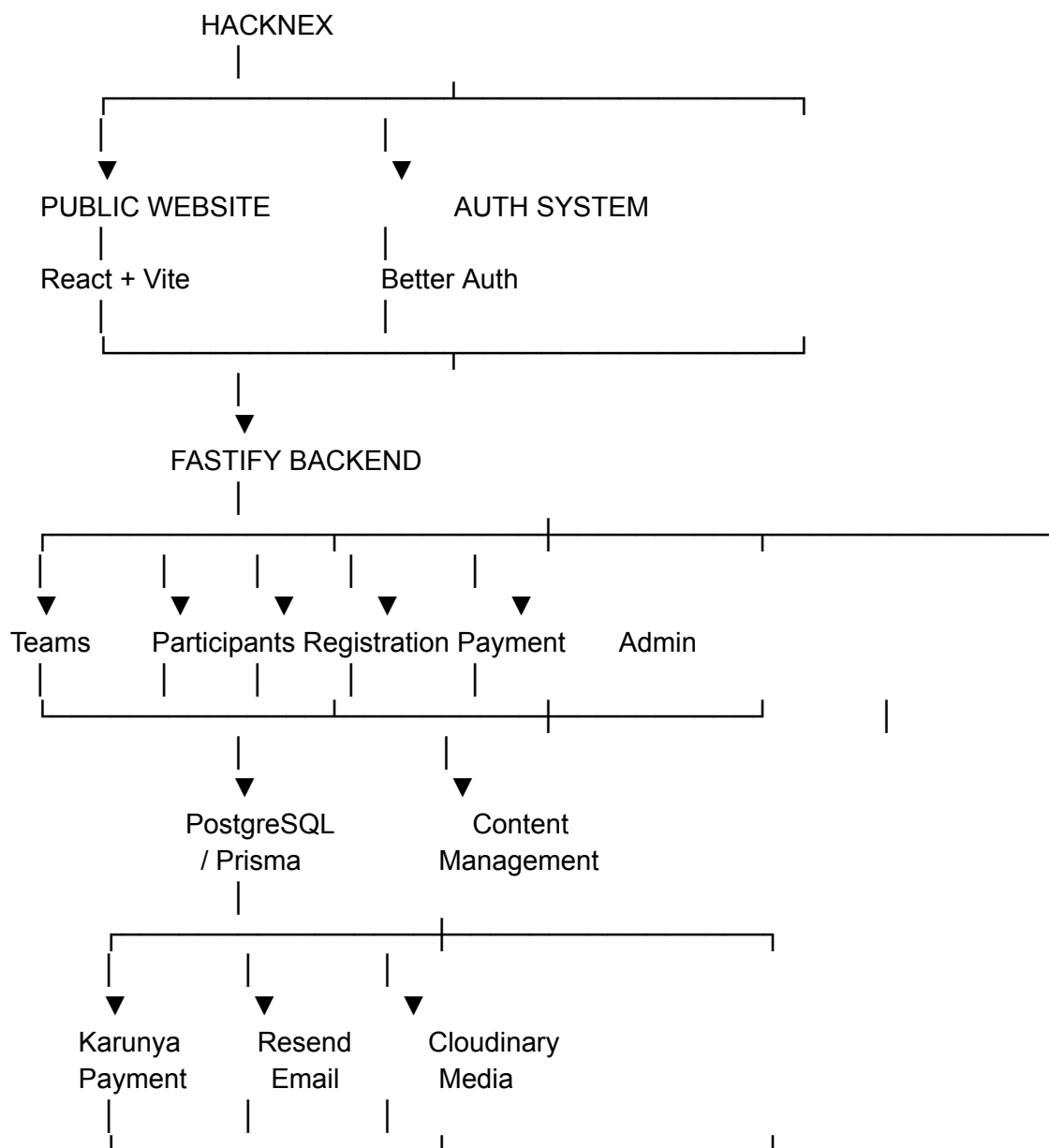
128. Architecture Decision Summary

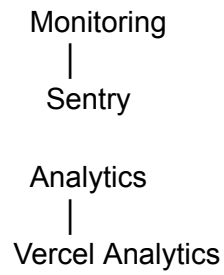
Decision	Final Choice
Architecture	Modular Monolith
Frontend	React + Vite
Language	TypeScript
Styling	Tailwind CSS
UI	shadcn/ui + 21st.dev
Animation	Framer Motion + CSS
Backend	Node.js + Fastify
ORM	Prisma
Database	PostgreSQL
Auth	Better Auth
Email	Resend
Media	Cloudinary
Frontend hosting	Vercel
Backend hosting	Render
Database hosting	Neon
Monitoring	Sentry
Analytics	Vercel Analytics
CI/CD	GitHub Actions
Payment	Karunya
Payment verification	Automatic
Cache	Not required initially
Redis	Deferred
Team size	Exactly 4
Fee	₹600/team
Refund	None

Admins	15
Admin roles	Admin only
Development	Solo
Expected users	1,500+

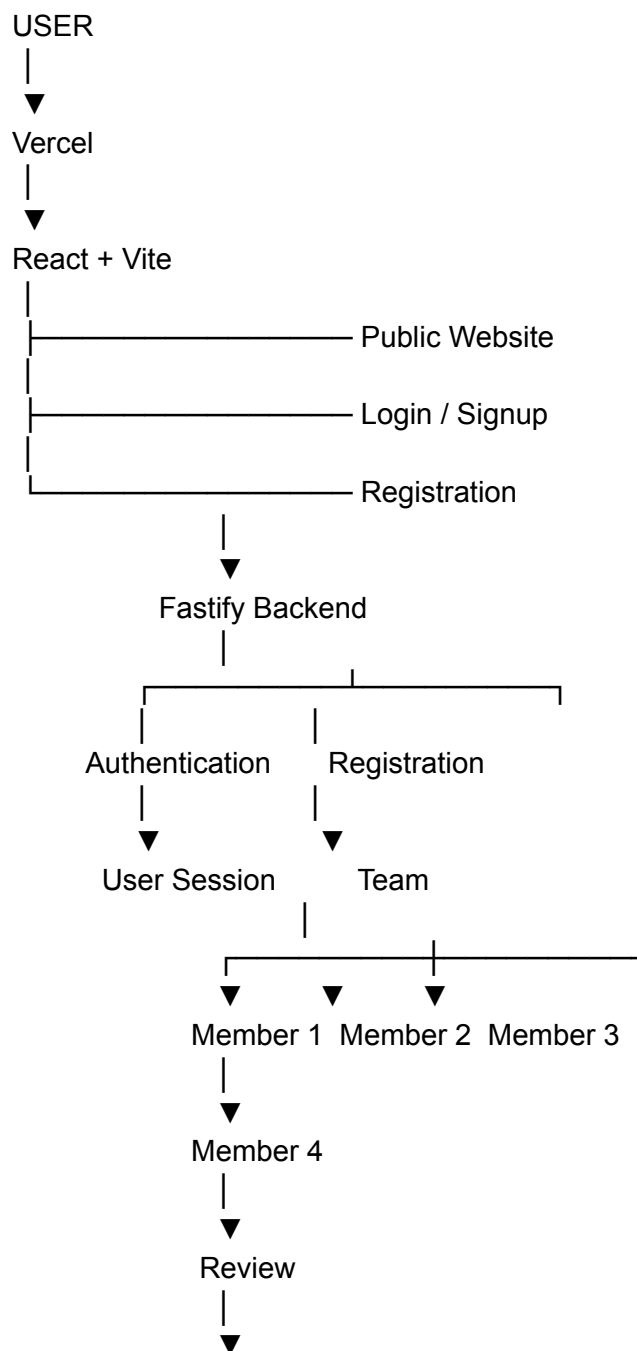
129. Final Architecture

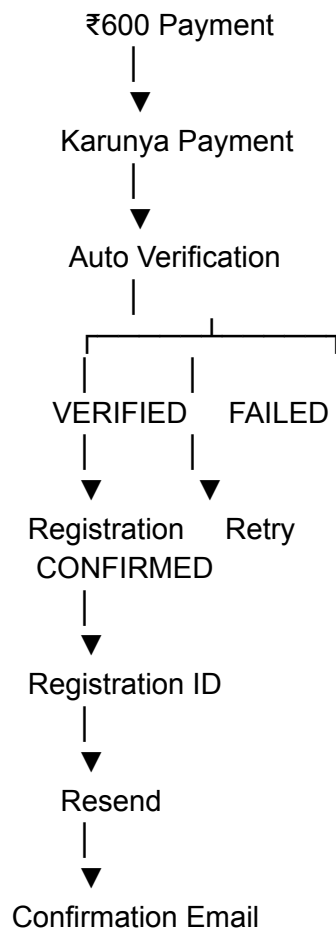
The final HackNEX system can be represented as:





130. Final End-to-End Architecture





VERY VERY IMPORTANT :

One architectural refinement I recommend

There is one thing I would **slightly change from the earlier architecture**: don't make the three non-captain participants create HackNEX accounts. Your workflow is cleaner if **only the captain authenticates**, enters all four participants, completes the ₹600 payment, and receives the confirmation email. The other three members simply receive whatever event information is subsequently communicated by the organizers.

That keeps the database and user-management system substantially simpler while still satisfying your requirement that **only logged-in/verified users can register**.